

KCoin, from zero

A step-by-step explanation of what you built, why a blockchain needs each part, and what happens from one click to a permanently recorded block.

This guide assumes nothing

You do not need to know what a wallet, key, transaction, block, validator, hash, database, API, or consensus rule is. Each term is explained before it is used.

[Start with a shared notebook](#)

[End with the complete network](#)

By the end, you will be able to explain:

- 1 where KCoin balances actually live;
- 2 how a wallet approves a payment without revealing its secret;
- 3 how three of four validators make a block final; and
- 4 how an offline validator catches up safely.

One program, several copies

KCoin is software: instructions that a computer follows. Several computers can run the same software and exchange messages.

Data

Information stored by a computer.

Program

A set of instructions a computer runs.

Message

Data sent from one program to another.

Network

Computers connected to exchange messages.

Why run more than one copy?

● Node 0

current

copy A

● Node 1

current

copy B

● Node 2

current

copy C

● Node 3

current

copy D

More copies remove one obvious weak point

✓ If one computer stops, the other copies can remain available.

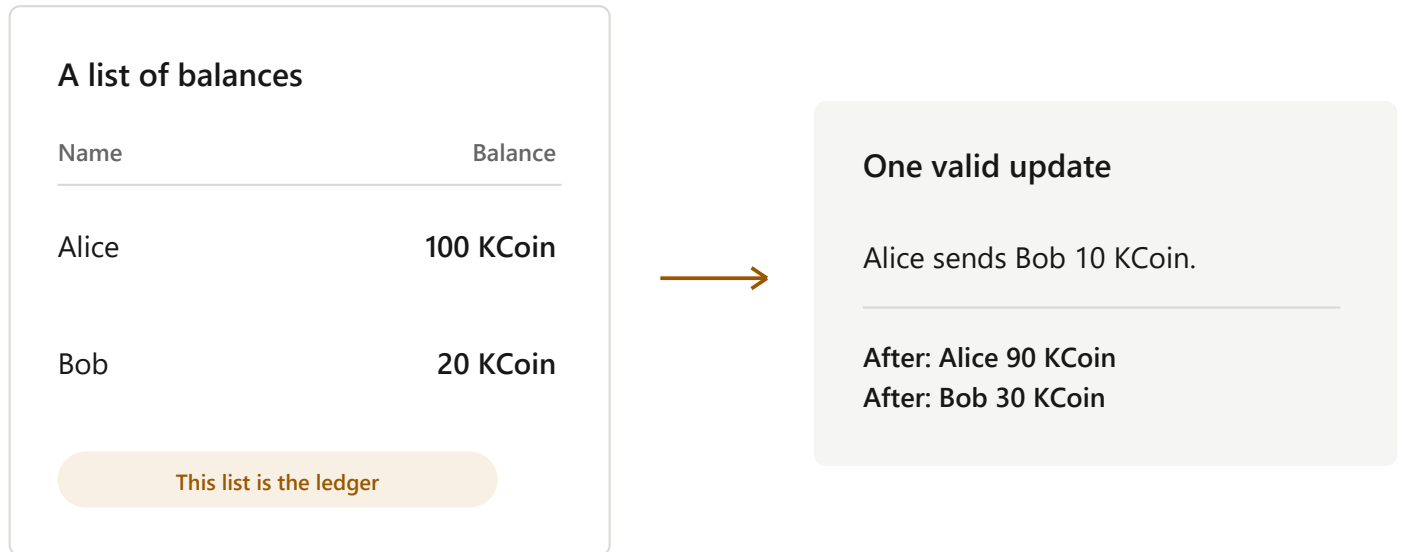
✓ Each checker can verify the same rules independently.

✗ The copies still need a rule for agreeing on one order.

That last problem - keeping independent copies in agreement - is where the blockchain design begins.

Begin with a record

Before money or blockchains, start with a simple fact: a computer can store a list of information.



New word

Ledger means the official record of balances and changes. KCoin's ledger says which address owns how much KCoin.

The most important starting point

A KCoin is not a picture, file, or object hidden inside a wallet. A balance is an entry in the shared ledger. The wallet only holds the secret that lets its owner request changes to that entry.

Copies can disagree

Messages arrive at different times. A node can be offline. A sender can make an invalid request. The network needs one shared answer.

1

Same request, different timing

Two nodes may hear transactions in a different order.

2

Missing information

An offline node may miss several completed updates.

3

Bad information

A forged or overspending transaction must not change the ledger.

Plain-English definition

A **blockchain** is a shared record built from ordered batches called blocks. The nodes follow fixed rules to decide which block becomes final, and each block points back to the previous one.

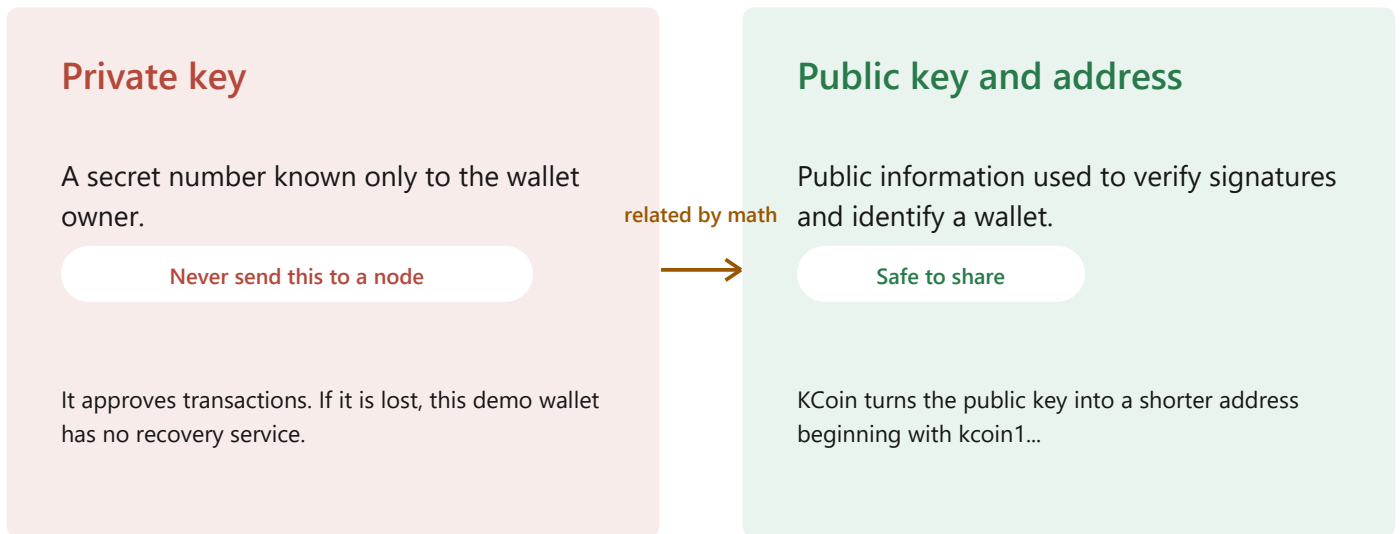
KCoin uses this design so four independent validators can maintain one history without one validator being the permanent master.

What this project focuses on

KCoin focuses on signed transactions, shared state, finality, persistent storage, and node recovery across four validators.

A wallet is a way to prove control

A wallet creates a pair of mathematically related keys. One stays secret. The other can be shared.



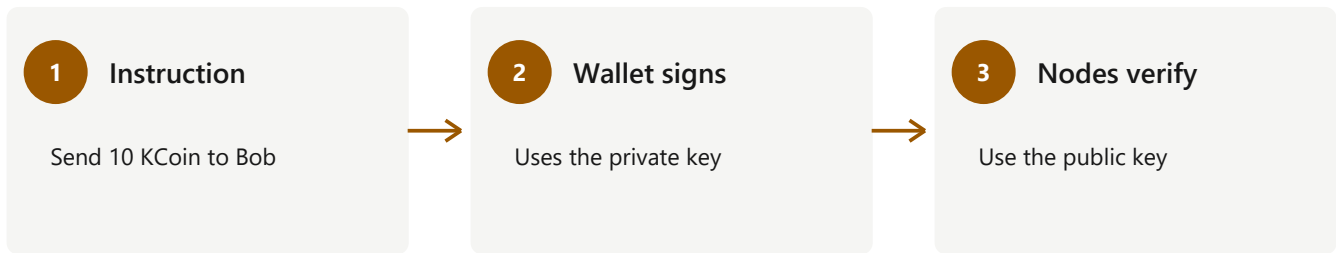
What the browser does

- 1 Create: generate the key pair inside the browser.
- 2 Back up: download the private key once before sending.
- 3 Use: keep the active private key only in browser memory.
- 4 Address: share kcoin1... to receive KCoin. A public display name is only a label; it cannot receive funds.

Remember: the wallet stores a secret, not coins. The ledger stores the balance associated with the address.

A signature proves who approved a message

A digital signature is mathematical evidence attached to exact data. Computers store that exact form as ordered numbers called bytes.



A valid signature proves

- ✓ the private key approved these exact bytes;
- ✓ the message was not changed afterward.

It does not prove

- ✗ the sender has enough KCoin;
- ✗ the transaction is allowed or final.

The name you may see in the code

Ed25519 is the established signature system KCoin uses. You do not need its mathematics to understand the design: the browser signs, and every validator independently verifies.

If Alice's signed message says 10 KCoin, an attacker cannot change it to 100 KCoin while keeping the signature valid.

A transaction is a signed instruction

It asks the ledger to change. It is not complete merely because the browser created or submitted it.

Example transfer

From	Alice's public key
Action	Send 10 KCoin
To	Bob's kcoin1... address
Nonce	Alice's next request number: 7
Expiry	Stop accepting after block 140
Chain ID	Only valid on kcoin-local-1

+ wallet signature

Why the extra fields exist

Nonce

A sequence number. Request 7 cannot be accepted twice, and request 6 cannot return later.

Expiry

A time limit measured by a future block number, so an old pending request eventually becomes invalid.

Chain ID

Prevents a signed request for one KCoin network from being replayed on another.

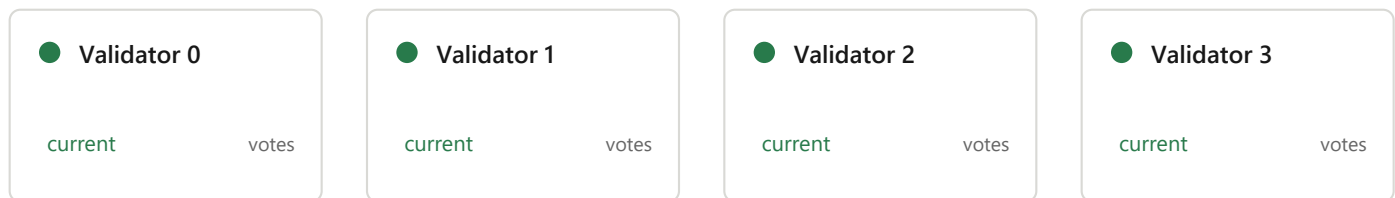
Before changing the ledger, every validator checks:

- ✓ correct signature and network;
- ✓ exact next nonce and not expired;
- ✓ valid recipient and positive amount;
- ✓ enough balance and no arithmetic overflow; and
- ✓ the same signed request has not already been accepted.

All checks happen before any ledger change. If one fails, balances and sequence numbers remain unchanged.

Four validators are independent checkers

A validator is a KCoin node allowed to vote on blocks. KCoin fixes four validator identities in its starting configuration.



Each validator does the same work

- 1 receive proposed block bytes;
- 2 verify every transaction and header rule;
- 3 execute the transactions on its own ledger copy;
- 4 compare the results each validator calculated independently;
- 5 vote only if the result is valid.

One proposer at a time

For each next block and retry, one validator is chosen by a simple rotation to assemble the candidate block. The proposer does not get the final say; the others verify it.

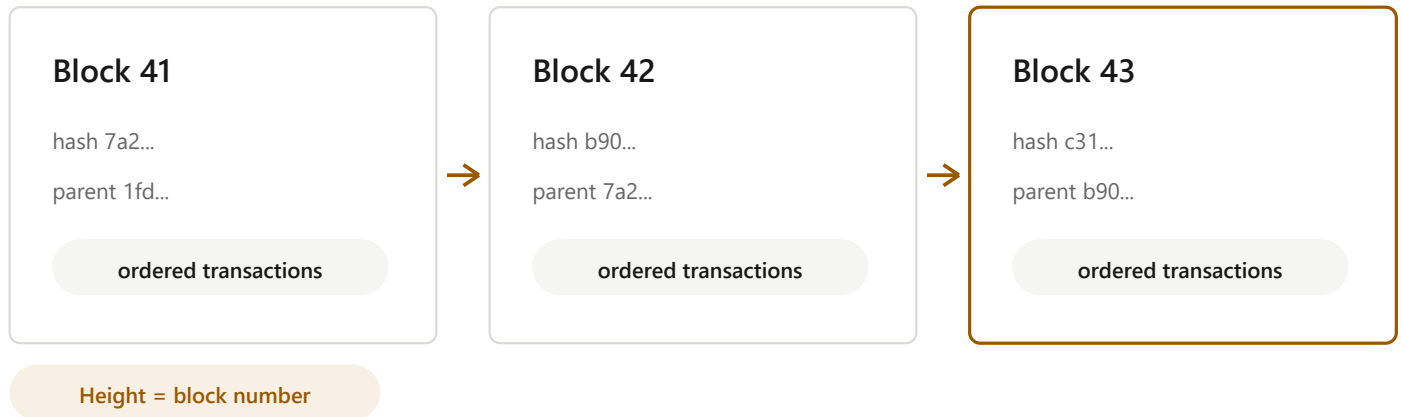
Observer

A read-only node serves the web app, relays transactions, and verifies completed blocks. It cannot vote.

Four is useful for the demo because three votes can make progress while one validator is unavailable.

A block is one agreed page

Instead of changing the official record for every request separately, KCoin orders transactions into blocks.



Hash

A short fixed-size fingerprint calculated from data. A tiny data change produces a different fingerprint.

Parent hash

The previous block's fingerprint. It links the pages in one exact order.

State root

A fingerprint of the complete ledger after the block. Nodes compare it to catch any different result.

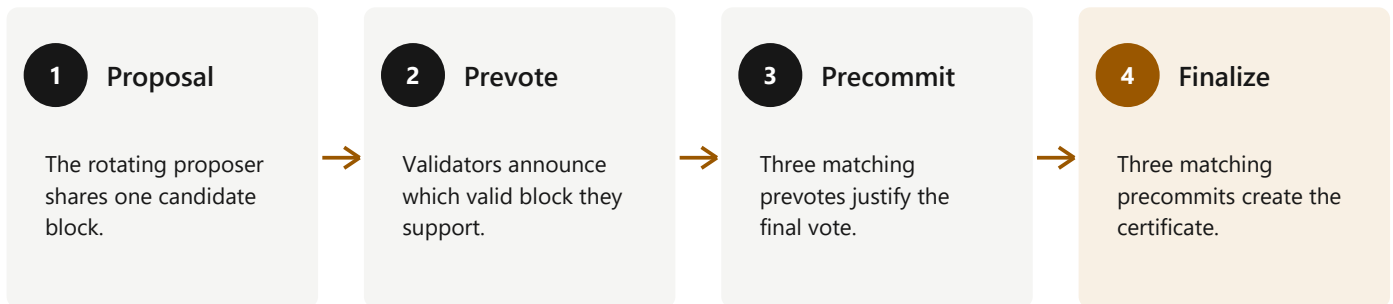
Transaction root

A fingerprint that commits to the block's transaction content and order.

The final validator signatures are stored beside the block in a **commit certificate**. Keeping them separate avoids trying to put signatures inside the data they sign.

Three matching votes make a block final

Consensus means the rules the validators follow to choose one next block. A quorum is the minimum matching votes required; KCoin's quorum is three.



3 of 4 available

A quorum still exists. Valid blocks can continue becoming final.

network continues

Only 2 of 4 connected

Neither side has three votes. The network pauses instead of guessing.

safe halt

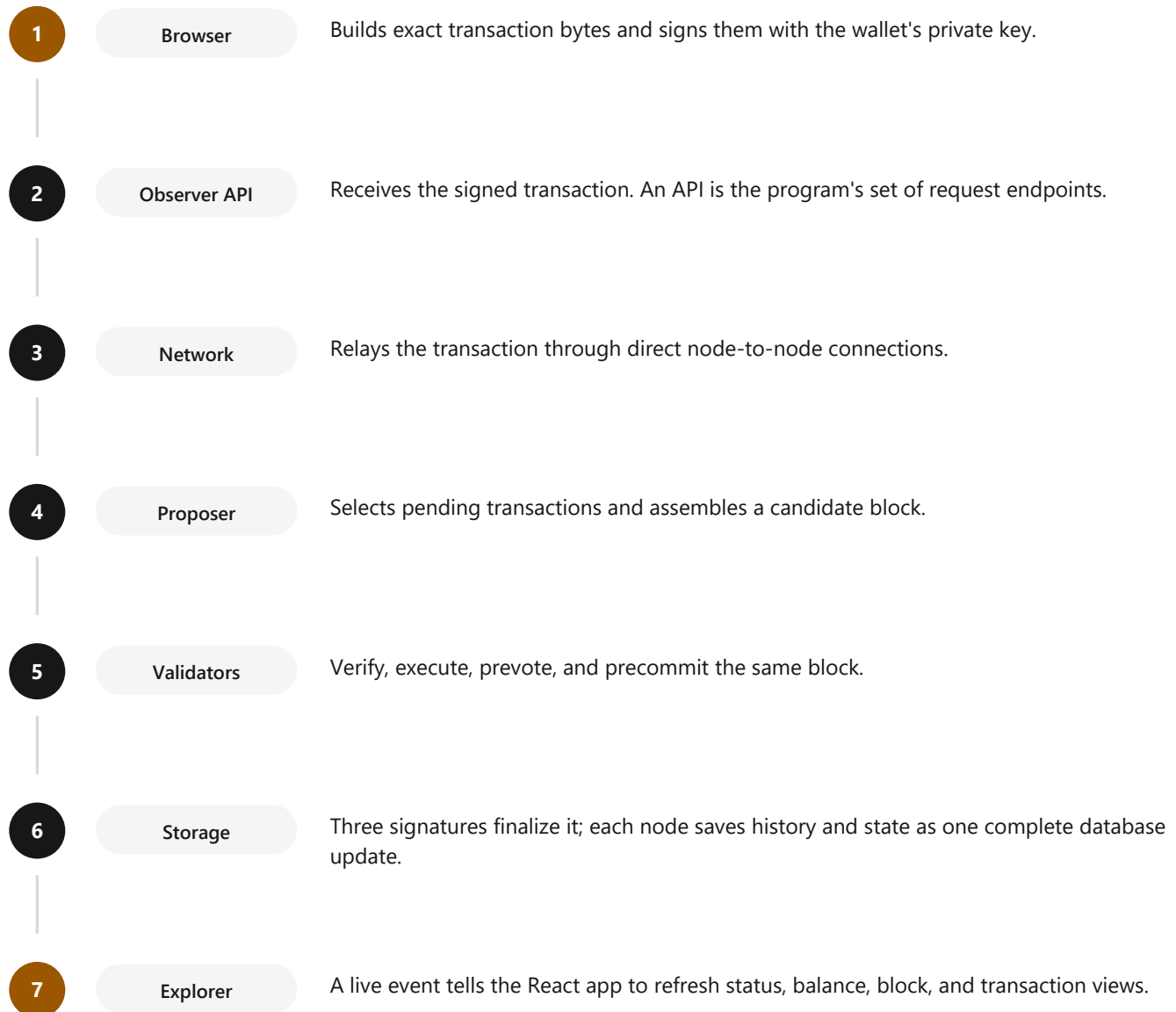
What final means

Once a block has a valid three-signature certificate, KCoin treats that block as the permanent block at that height. It has no longest-chain reorganization rule.

Safety guardrail: a validator that has supported a block can become locked on it. It cannot switch to a conflicting block unless a newer three-vote proof makes that switch safe.

What happens when you click Send

The visible button starts a chain of work across the browser, observer, validators, disks, and explorer.



The browser may first show **pending**. The transfer is **finalized** only after the observer receives, verifies, and stores the certified block.

Earn is how new KCoin enters the ledger

Issuance means creating new units according to fixed rules. KCoin uses a simple arithmetic challenge so issuance is easy to demonstrate.



The fixed supply schedule

0-20k	20-40k	40-60k	60-80k	80-100k
100 KCoin	50 KCoin	25 KCoin	10 KCoin	5 KCoin

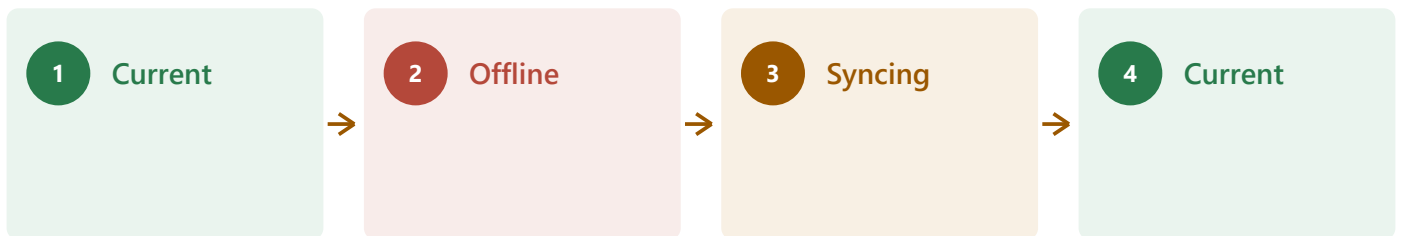
One KCoin equals 1,000,000 atoms, so the ledger uses whole numbers instead of rounding decimals. Supply stops exactly at 100,000 KCoin. No KCoin is assigned at startup, and there is no transaction fee.

What this challenge is - and is not

- ✓ A predictable, rule-based way to demonstrate issuance.
- ✓ The first valid claim included in a finalized block wins.
- ✗ Not mining, proof of work, or a claim of bot-resistant fairness.

An offline validator must prove its way back

The returning node does not trust another node's balance summary. It downloads finalized history and verifies each missing step.



During Syncing, the validator checks every missed block

- 1 Certificate** Three authorized validator signatures prove finality.
- 2 Parent link** The block follows the exact block it already trusts.
- 3 Transactions** Every signature, nonce, balance, challenge, and amount is valid.
- 4 Roots** Re-execution produces the stored transaction root and state root.

Why it cannot vote immediately

A lagging validator enters non-voting Syncing mode. It rejoins consensus only after its finalized height, block hash, and state root match the current network.

The network must survive a crash

Memory disappears when a program stops. Durable storage means writing important data to disk so restart can recover it.

Database

An organized file used to store and retrieve data. Every KCoin node owns a separate SQLite database.

Atomic commit

The block, certificate, balances, indexes, and tip appear together - or none of them appear.

Canonical history

The original finalized block and certificate bytes. Derived views can be rebuilt from this source of truth.

Crash-safe signing

Before broadcasting a proposal or vote, a validator stores the exact decision and signature. After restart, it reuses an identical signature and refuses a conflicting request for the same slot.

Rebuildable views

verify replays history and checks the database.
reindex rebuilds balances, transactions, names, leaderboard data, and metadata from canonical history.

SQLite's own write-ahead log protects database writes. KCoin also has a separate consensus safety record that protects what a validator is allowed to sign.

The explorer makes invisible work visible

The frontend is not decoration around the blockchain. It is how a person creates requests and inspects the network's real state.

Wallet

Create or import a key, back it up, see balance, name the address, and send KCoin.

Earn

See the active arithmetic challenge, reward, answer field, and claim status.

Blocks

Inspect finalized heights, hashes, roots, proposer information, and certificates.

Transactions

Follow pending or finalized requests and open sender or recipient addresses.

Validators

See all four nodes as online, voting, offline, syncing, or current.

Ownership

Compare balances through a packed-circle map and an accessible leaderboard.

The primary demo path

Create wallet -> Earn -> Send -> Explore -> Stop validator -> Recover

Historical data comes through normal API requests. Finalized-block and validator events update the interface live without pretending the browser decides finality.

What you actually built

The strongest project story is not 'I invented cryptography.' It is that you integrated difficult systems into one testable, understandable product.

- 1 Protocol** Transaction types, canonical bytes, addresses, validation rules, supply, blocks, roots, certificates.
- 2 Consensus** Deterministic proposal and voting state machine, quorum, proposer rotation, locks, timeouts.
- 3 Node** Ledger actor, mempool, peer-to-peer messaging, synchronization, storage, REST, SSE, health, metrics.
- 4 Frontend** Memory-only browser wallet, signing adapter, explorer, validator recovery view, ownership map.
- 5 Evidence** Adversarial tests, simulations, integration tests, Docker devnet, demo media, diagrams, threat matrix.

Established tools you used correctly

Ed25519 for signatures; BLAKE3 for fingerprints; Borsh for exact binary encoding; libp2p and QUIC for networking; SQLite for persistence; Axum for HTTP; React and WebCrypto in the browser.

Using established primitives is good engineering. The authored value is the protocol, state transitions, safety boundaries, recovery behavior, interface, and evidence around them.

Where to look in the repository

Protocol	crates/kcoin-protocol
Consensus	crates/kcoin-consensus
Node and recovery	crates/kcoin-node
Frontend	web

You can now explain KCoin in 30 seconds

Use the short version first. Add technical names only when the listener wants more detail.

KCoin is a small blockchain I built in Rust with a React explorer. A browser wallet signs requests, four validators independently check them, and three matching votes make a block final. Each node stores and can replay the certified history. If one validator goes offline, the other three continue; when it returns, it verifies every missed block before voting again.

Mini glossary

Ledger official balances and history

Wallet keys used to prove control

Address public destination for KCoin

Signature proof that exact data was approved

Transaction signed request to change the ledger

Block ordered batch added to history

Hash data fingerprint

Validator node allowed to vote

Consensus rules for choosing one next block

Quorum enough votes; three in KCoin

Finality accepted permanently at that height

Syncing verifying missed finalized history

One last check: the wallet approves a transaction; validators decide whether it is valid and final; the explorer shows the result. Those are three separate responsibilities.